

VeritasRAG

Complete Project Theory Guide

Written to be understandable by anyone, regardless of technical background

Project	VeritasRAG — an AI chatbot with built-in safety and accuracy checks
Prepared By	
Date	

Contents

1. Introduction — What Problem Are We Solving?
2. Two Ideas You Need First: Chatbots and RAG
3. The Big Picture — How a Message Travels Through the System
4. Layer One: The Input Filter
5. Layer Two: Giving the Chatbot Real Knowledge
6. Layer Three: The Faithfulness Checker
7. Layer Four: Content Relevance Checking (Planned)
8. A Complete Walkthrough — Following One Message Step by Step
9. The Tools and Technologies We Used
10. How We Test the System
11. What Isn't Perfect Yet
12. How We Worked — Our Process
13. Glossary of Terms
14. Conclusion

1. Introduction — What Problem Are We Solving?

Imagine hiring a new employee to answer questions from staff or customers. This employee is fast, polite, and knows a huge amount about the world in general — but they have never seen your company's actual policies, they might occasionally make something up rather than admit they don't know, and they have no filter for a rude or dangerous question. That, roughly, is what a general-purpose AI chatbot is like straight out of the box.

VeritasRAG exists to fix exactly those three problems. It takes a general-purpose AI chatbot and wraps it in three layers of protection: one layer that checks what the user is asking before the AI ever sees it, one layer that feeds the AI real company information instead of relying on guesswork, and one layer that double-checks the AI's answer afterward to catch anything it might have made up.

This document explains how each of those layers actually works, in plain language, with no assumed background in programming or artificial intelligence. Technical terms are explained the first time they appear, and a full glossary is included at the end.

2. Two Ideas You Need First: Chatbots and RAG

2.1 What is a chatbot, technically?

At its core, a chatbot is a program that sends your typed message to a large language model (LLM) — an AI system trained on enormous amounts of text — and displays whatever text the LLM sends back. Well-known examples of LLMs include the models behind ChatGPT and similar assistants. VeritasRAG uses an LLM accessed through a service called OpenRouter, which acts as a single doorway to many different AI models from different companies.

2.2 What does 'RAG' mean?

RAG stands for Retrieval-Augmented Generation. Retrieval means looking something up. Generation means the AI writing a response. Put together, RAG simply means: before the AI writes its answer, first go and fetch some real, relevant information, and hand that information to the AI so it can use it.

In everyday terms

It's the difference between asking a friend to answer a question from memory, versus handing them the actual company handbook open to the right page first, and asking them to answer using only what's written there.

VeritasRAG is a RAG chatbot: instead of letting the AI answer purely from its own general training, it hands the AI real content from a knowledge base — a document containing the actual facts the chatbot is supposed to know — before asking it to answer.

3. The Big Picture — How a Message Travels Through the System

Every message a user sends passes through a chain of checks before a reply is shown. Nothing happens in a single step — there is a whole pipeline, and each part of the pipeline has one specific job.

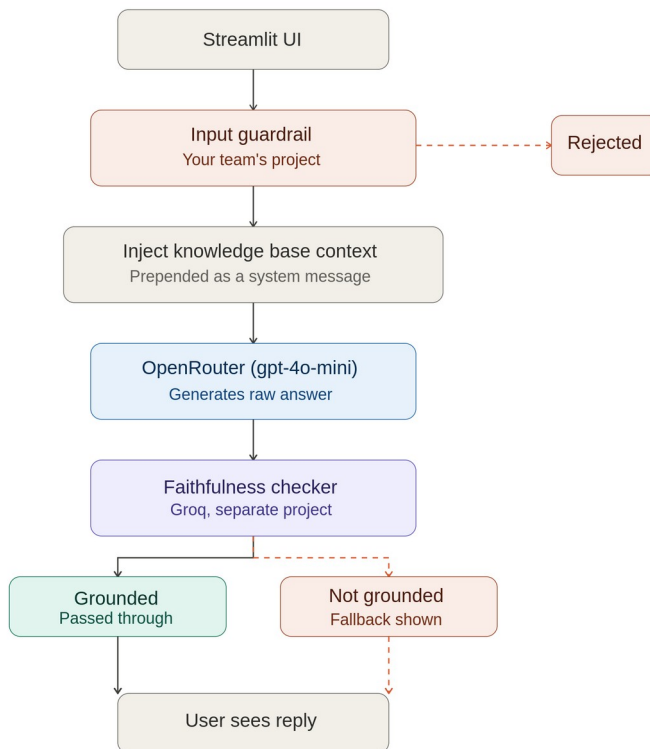


Figure 1 — The complete journey a message takes, from the user's screen to the reply they see

Reading the diagram from top to bottom: the user types a message. It first meets the input filter — if the message is unsafe, it is stopped immediately and the AI never sees it. If it passes, relevant knowledge-base content is attached to the message before it is sent to the AI. The AI writes a reply. That reply is then independently checked by the faithfulness checker before the user ever sees it — if the checker cannot confirm the reply is genuinely supported by the knowledge base, the user is shown a safe fallback message instead.

4. Layer One: The Input Filter

The input filter's only job is to look at what the user typed and decide: is this safe to pass on? It never generates any part of the reply itself — it is purely a gatekeeper.

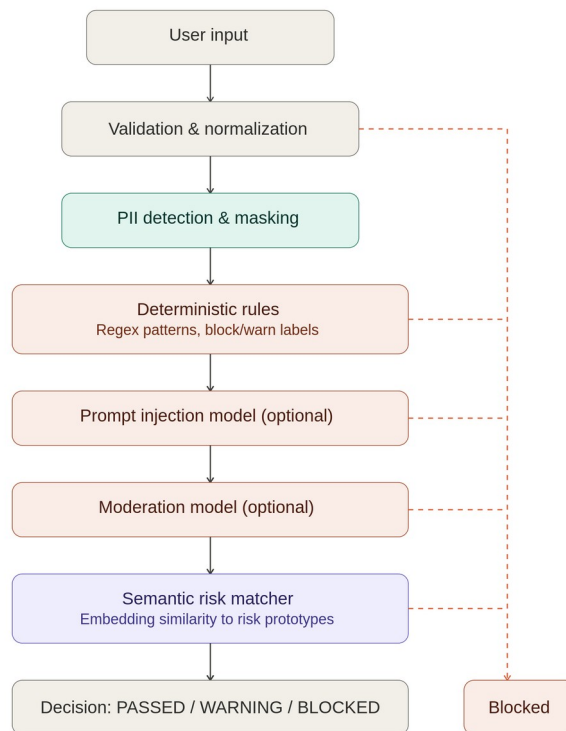


Figure 2 — The input filter's checks, run in order

4.1 What it checks for, in plain terms

- Basic sanity checks: is the message empty, or absurdly long?
- Private information: does the message contain something like an email address or phone number that shouldn't be passed along unprotected? If so, it is automatically hidden (for example, replaced with [EMAIL]) before continuing — the conversation still works, just without exposing that detail.
- Known attack patterns: does the message try to trick the AI into ignoring its instructions, pretending to be an unrestricted version of itself, or contain hidden malicious code?
- Dangerous requests: does the message ask for help with violence, self-harm, weapons, fraud, or similarly harmful topics?
- A final "does this sound like an attack" check: rather than just matching exact phrases, this stage compares the meaning of the message against a list of known harmful examples, so that reworded or disguised attempts are still caught.

4.2 Avoiding a common mistake: blocking innocent questions

A filter that is too aggressive is almost as much of a problem as one that is too weak — imagine asking "what is phishing awareness?" for a training course, and being told your question is itself an attack. VeritasRAG's filter specifically checks for this: if a message is clearly educational or asking about a topic defensively (for example, using words like "what is", "explain", or "prevent"), it is treated differently from a message that is actually trying to do the harmful thing itself.

This distinction is not automatic or perfect, however — during testing, this exact filter occasionally still misjudged an ordinary question as suspicious, which is discussed further in Section 11.

5. Layer Two: Giving the Chatbot Real Knowledge

Once a message passes the input filter, it still needs to actually be answered correctly. This is where the knowledge base comes in — a plain text document containing the real facts the chatbot should know (for example, a company's IT support policies).

Before the AI is asked to write a reply, the chatbot secretly attaches the entire knowledge base document to the conversation, along with an instruction along the lines of: "answer using only the information below." This is the "Generation" side of RAG mentioned in Section 2 — real information is handed over before the AI writes anything.

In everyday terms

It's like a call-center employee getting handed a sticky note with the exact policy answer right before they pick up the phone, instead of trying to remember it from a training session months ago.

This step alone dramatically improves accuracy for domain-specific questions, but it has one important limitation worth understanding: for now, the entire knowledge base document is attached to every single question, in full. That works fine while the document is small, but would become slow and expensive if the knowledge base grew into a large library of documents — see Section 11 for how this would need to change.

6. Layer Three: The Faithfulness Checker

Handing the AI the right information (Layer Two) makes a correct answer likely, but not guaranteed — AI language models can still occasionally add a plausible-sounding detail that isn't actually true or isn't actually in the source material. This behavior is often called "hallucination." The faithfulness checker's entire job is to catch this after the fact, before the user ever sees the answer.

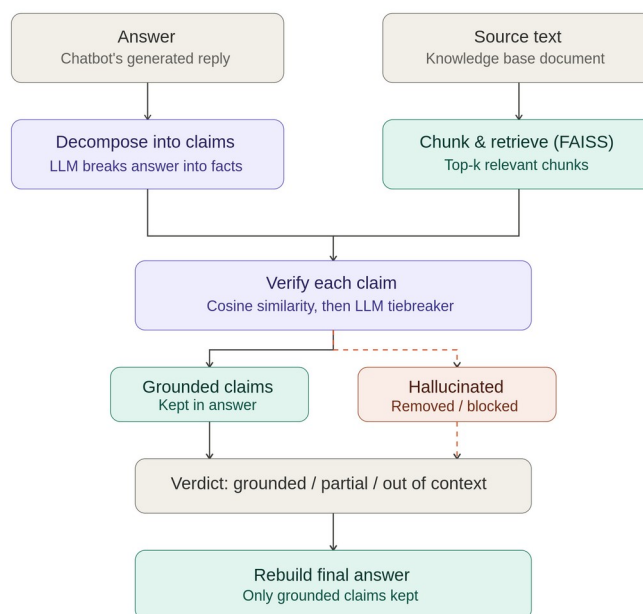


Figure 3 — How one answer gets fact-checked, claim by claim

6.1 How it checks an answer, step by step, in plain terms

- First, the answer is broken down into individual, single-fact sentences — a bit like a teacher marking an essay by underlining each separate claim to check one at a time, rather than judging the whole paragraph as one block.
- Each individual claim is then compared against the actual knowledge base content, to see how closely related they are in meaning.
- If a claim is clearly well-supported by the source material, it's accepted immediately. If a claim is clearly unrelated to anything in the source material, it's rejected immediately. Only in the genuinely unclear, in-between cases does the system make one extra check by directly asking the AI, "is this specific claim actually backed up by this specific source text — yes or no?"
- Finally, a verdict is produced: was every claim in the answer verified (Fully Grounded), was only some of it verified (Partially Grounded), or was none of it verified (Out of Context)?

6.2 What happens to a partly-wrong answer

If some claims are verified and some aren't, the unverified parts are not simply deleted, which would leave a choppy, half-finished-looking sentence. Instead, the AI is asked a second time to write a completely new, natural-sounding answer — but this time using only the parts that were actually verified. If nothing in the answer could be verified at all, the user is shown a clear, honest fallback message rather than any invented information.

7. Layer Four: Content Relevance Checking (Planned)

This layer does not exist yet — it is planned, designed, but not yet built. It solves a different problem to the faithfulness checker in Layer Three, and the difference matters.

The faithfulness checker (Layer Three) checks an answer after the AI has already written it. Content relevance checking would instead check the question itself, before the AI is ever asked to answer — essentially asking "is this question even about something our knowledge base covers?" before spending the time and cost of generating and then fact-checking a full answer.

In everyday terms

It's the difference between a receptionist saying "that's not something we handle here, try the department down the hall" immediately (relevance checking), versus letting someone fully explain their issue to the wrong department first, only to be told afterward that it wasn't the right place (the current system, without this layer).

8. A Complete Walkthrough — Following One Message Step by Step

To make all of the above concrete, here is exactly what happens when a real user asks a real question: "What is the password policy?" — assuming a knowledge base that contains a company's IT support documentation.

#	Who / What	What Happens
1	User	Types "What is the password policy?" into the chat box and presses send.
2	Input Filter	Reads the message. No unsafe patterns, no attack attempts, no private information detected. Decision: safe to continue.
3	Chatbot	Loads the knowledge base document from disk and attaches its full content to the conversation, along with an instruction to answer only using that content.
4	AI Model (via OpenRouter)	Reads the attached knowledge base content and writes an answer — for example, describing the minimum password length and expiry period.
5	Faithfulness Checker	Breaks that answer into individual factual claims and checks each one against the same knowledge base content.
6	Faithfulness Checker	Finds that every claim in the answer is genuinely supported by the source document. Verdict: Fully Grounded.
7	Chatbot	Displays the AI's original answer to the user, unchanged, since nothing needed to be corrected or removed.
8	User	Sees the accurate, verified answer on screen.

For comparison: if the user had instead asked something completely unrelated, like "what's today's weather forecast?", Step 6 would instead find that no claim in the AI's answer is genuinely supported by the knowledge base — and the user would see a clear message explaining that the chatbot doesn't have reliable information on that topic, rather than a guessed answer.

9. The Tools and Technologies We Used

None of this was built from scratch — like most modern software, it stands on well-established tools. Here is what each one does, in plain terms.

Python	The programming language the entire project is written in.
Streamlit	A tool for quickly building a simple chat-style web page, used here only as a test interface — not the final product.
OpenRouter	A service that provides access to AI language models from different companies through one connection, used here to generate the chatbot's actual replies.
Groq	A separate AI service used specifically by the faithfulness checker to do its fact-checking work — a different provider from the one generating the main reply.
Embeddings / sentence-transformers	A technique for converting a piece of text into a list of numbers that represents its meaning, so that two pieces of text can be compared for similarity by a computer, not just by matching exact words.
FAISS	A tool for quickly searching through many pieces of text to find the ones most similar in meaning to a given question — used to find the relevant part of the knowledge base.
Presidio (optional)	A specialized tool for detecting private information such as names and ID numbers in text, used as an optional upgrade to the simpler built-in detection.

10. How We Test the System

The system is tested by actually using the chatbot and checking whether it behaves as expected — not just reading the code. A written suite of 30 test cases covers seven different areas: whether correct answers are grounded in real content, whether unsafe messages are blocked, whether private information is hidden, whether harmless questions are wrongly blocked, unusual edge cases like empty messages, whether ordinary conversation (like saying "hi") is handled naturally, and whether the whole system works correctly end to end, including how quickly it responds.

11. What Isn't Perfect Yet

Being upfront about current limitations is part of understanding how the system really works — no system like this is flawless, and knowing exactly where the rough edges are matters more than pretending they don't exist.

- The faithfulness checker (Layer Three) is very good at checking facts stated directly in the knowledge base, but it can struggle with a question that requires combining two separate facts together — for example, if the document separately mentions weekday hours and an emergency line, a question about weekend availability might not be recognized as answerable, even though a person reading both facts together clearly could answer it.
- The entire knowledge base is currently attached in full to every single question (see Section 5). This works well for a small document but would need to change to only attaching the most relevant portion, once the knowledge base grows larger.
- Content relevance checking (Layer Four) is designed but not yet built.

- The input filter (Layer One) occasionally misjudges an ordinary, harmless question as suspicious when an additional optional detection layer is switched on — this optional layer is currently switched off as a result, while a more reliable version is worked out.
- If the faithfulness checker is temporarily unavailable or produces an error, the current behavior is to show the AI's original, unchecked answer rather than blocking it outright. Whether this is the right choice, or whether it should instead show nothing at all in that situation, is a decision still to be finalized.

12. How We Worked — Our Process

The team works in one-week cycles, called sprints, following a common project-management approach called Scrum. At the start of each week, the previous week's completed work and the coming week's planned work are both reviewed together with the supervisor, using a shared tracking document. Version control (using a tool called Git) keeps a complete history of every change made to the project, and significant pieces of work are developed on their own separate copy of the project (called a branch) before being merged back into the main version, to avoid unfinished work disrupting anyone else.

13. Glossary of Terms

AI / Artificial Intelligence	Software designed to perform tasks that normally require human judgement, such as understanding and generating natural-sounding language.
LLM (Large Language Model)	The specific type of AI system that reads and generates text, trained on very large amounts of writing. This is the technology behind chatbots like ChatGPT.
RAG (Retrieval-Augmented Generation)	Giving an AI real, relevant information to read before it answers, instead of relying only on what it already learned during training.
Knowledge base	A document (or collection of documents) containing the real, specific facts a chatbot should answer from.
Hallucination	When an AI states something confidently that isn't actually true or isn't actually supported by its source material.
Grounded / Grounding	An answer is "grounded" when it is genuinely backed up by real source material, rather than invented.
Embedding	A way of converting text into numbers that represent its meaning, so a computer can measure how similar two pieces of text are.
PII (Personally Identifiable Information)	Details that could identify a specific person, such as an email address, phone number, or ID number.
Prompt injection	An attempt to trick an AI into ignoring its instructions by disguising a command as part of an ordinary message.
Chit-chat	Casual conversational messages like greetings or thanks, which don't need to be checked against the knowledge base.
Sprint	A fixed period of time (one week, in this project) during which a specific, agreed set of work is completed.

14. Conclusion

VeritasRAG takes a general-purpose AI chatbot and makes it safer and more trustworthy through three layers working together: a filter that stops harmful input before it's ever seen, a knowledge base that gives the AI real facts to work from instead of guesswork, and an independent fact-checker that reviews every answer before it reaches the user. Each of these three layers was built as a separate, self-contained piece — meaning the disposable test chatbot used to demonstrate the system can later be swapped out for a different application entirely, while the safety and accuracy layers underneath continue to work exactly the same way.